



Statistical Methods & Machine Learning 1

Class #12

John Kubale

November 19, 2024

Outline

- Review
- Variable Transformation
 - Transforming Outcome Variables
 - Interpreting transformed variables
 - Polynomials
 - GAMs
- Model Selection
 - Background
 - Criterion-Based Procedures
 - Forward, Backward, Stepwise Selection



Review

Collinearity

- Eigenvalues, condition numbers, and kappa
- Variance inflation factor

Collinearity

- Eigenvalues – we're concerned about any predictors with eigenvalues close to 0 as a value of 0 indicates perfect collinearity
- Condition numbers (and kappa) give us another way of assessing eigenvalues

Collinearity

- What does the formula below actually tell us?
- $condition\ number = \sqrt{\max(eigenvalue) / eigenvalue_i}$
 - The higher the condition number the greater the difference between your largest eigenvalue and the eigenvalue in the denominator
- Kappa is simply your largest condition number or
 - $kappa = \sqrt{\max(eigenvalue) / \min(eigenvalue)}$
 - We should investigate if $kappa > 30$
 - Does $kappa > 30$ mean there is definitely collinearity?

Collinearity

No! Large condition numbers and/or a kappa > 30 only tells us that at least one of our eigenvalues is much smaller than our largest eigenvalue

This could be due to collinearity – denominator of condition number formula is close to 0 leading to large condition number/kappa

Could also be driven by very large maximum eigenvalue – numerator of condition number is large leading to large condition number/kappa

Variance inflation factor

- Should look at what potential collinearity is doing to our variance

- Do we need to drop a variable?
 - If so, which one(s)?

$$VIF(\hat{\beta}_j) = \frac{1}{1 - R_{X_j|X_{-j}}^2}$$

- Rule of thumb in the social sciences is predictors with $VIF > 5$ or 10 are likely problematic
- Can also roughly interpret $\sqrt{VIF}$ as how much larger we would expect our standard error for that variable to be compared to if there were no collinearity
 - E.g., $VIF = 307.4 \rightarrow \sqrt{307.4} = 17.5$ would expect standard error to be 17.5 times larger than if no collinearity $\rightarrow$ while useful can't be applied as a correction because didn't actually observe orthogonal data



Variable Transformation

Why variable transformation?

Why transform?

- Can improve model fit
 - Linear relationship
 - Prediction
- Potential remedy when important OLS assumptions are violated
 - Zero error mean
 - Constant error variance

Why not transform?

- Interpretation difficulty (e.g., logwage)
- There are more flexible tools available

We will examine transformation through simulated data

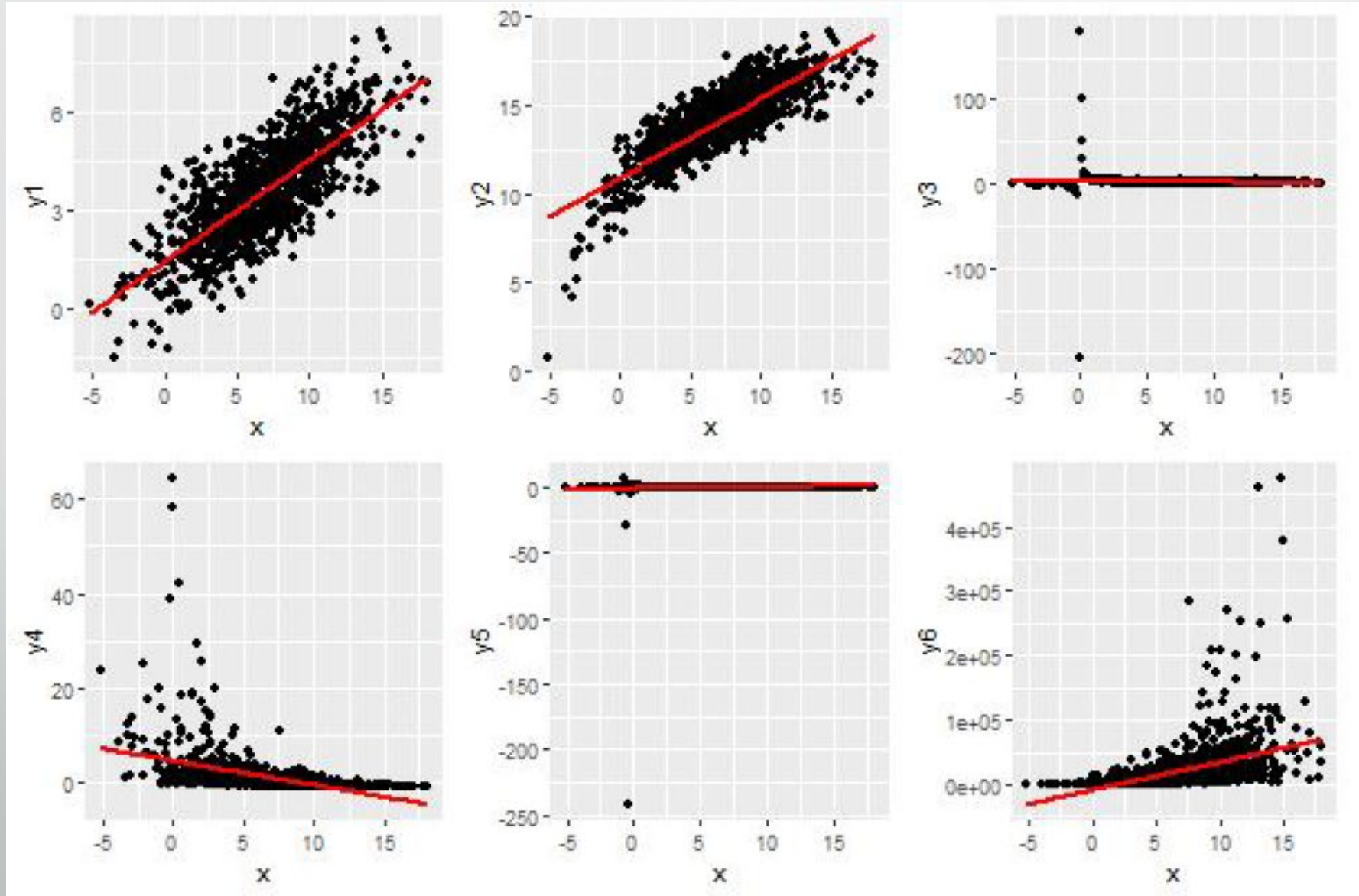
Example: Simulated Data

```
id<-c(1:1000)
set.seed(9); x<-rnorm(1000,7,4)
set.seed(123); e<-rnorm(1000,0,1)

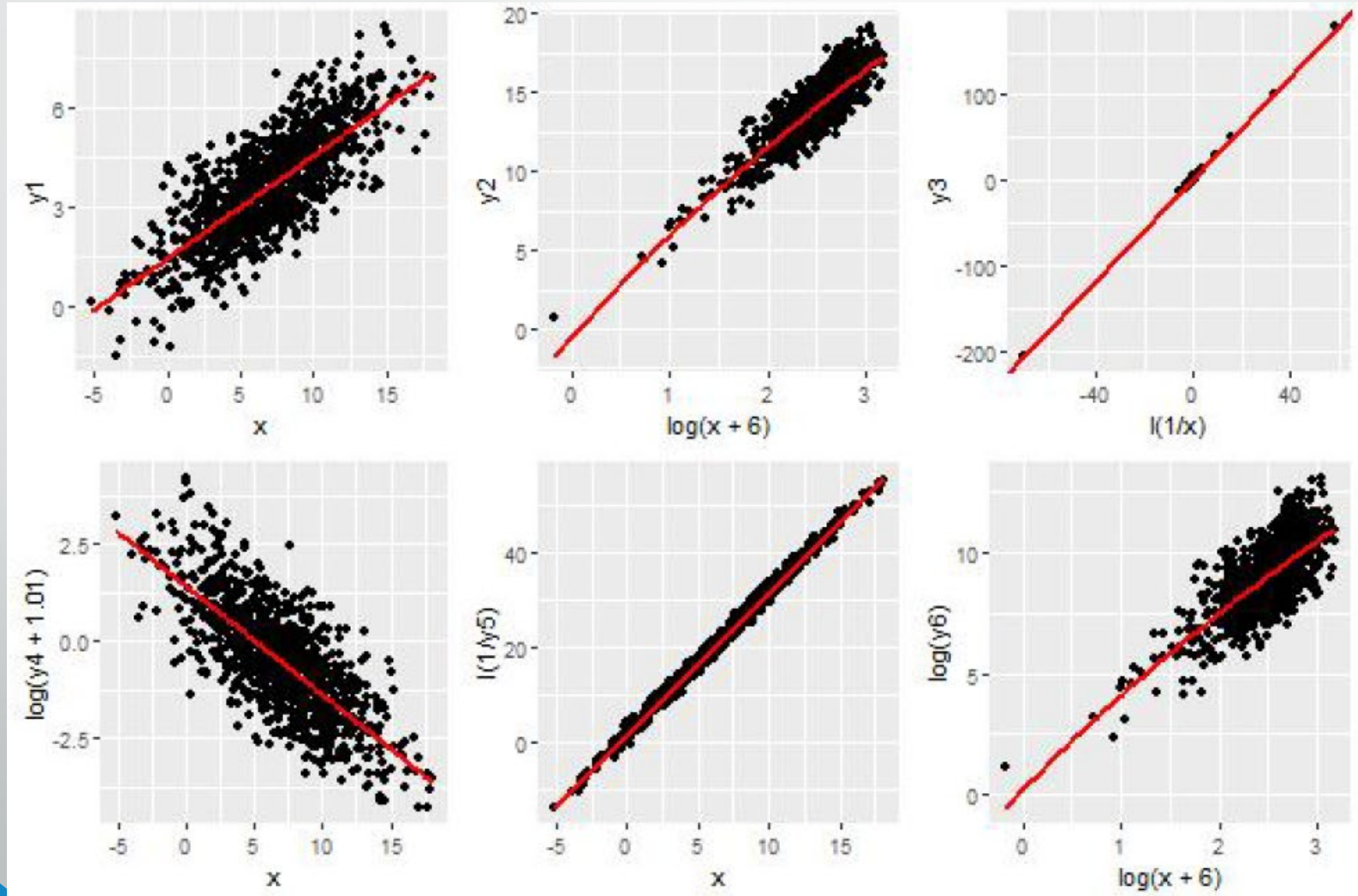
y1<-1.5+0.3*x+e
y2<-1.5+5*log(x+6)+e
y3<-1.5+3*1/x+e
y4<-exp(1.5-0.3*x+e)-1
y5<-1/(1.5+3*x+e)
y6<-exp(1.5+3*log(x+6)+e)
```

1. What is being done in the data? Relationship between x and each of y_1 to y_6 ?
2. What would happen to the residuals and fitted values if we are to model each of y_1 to y_6 onto x in simple linear regression?

Example: Scatterplots Without Transformation



Example: Scatterplots with Transformation





Transforming Outcome Variables

Box-Cox Family of Transformation on Y

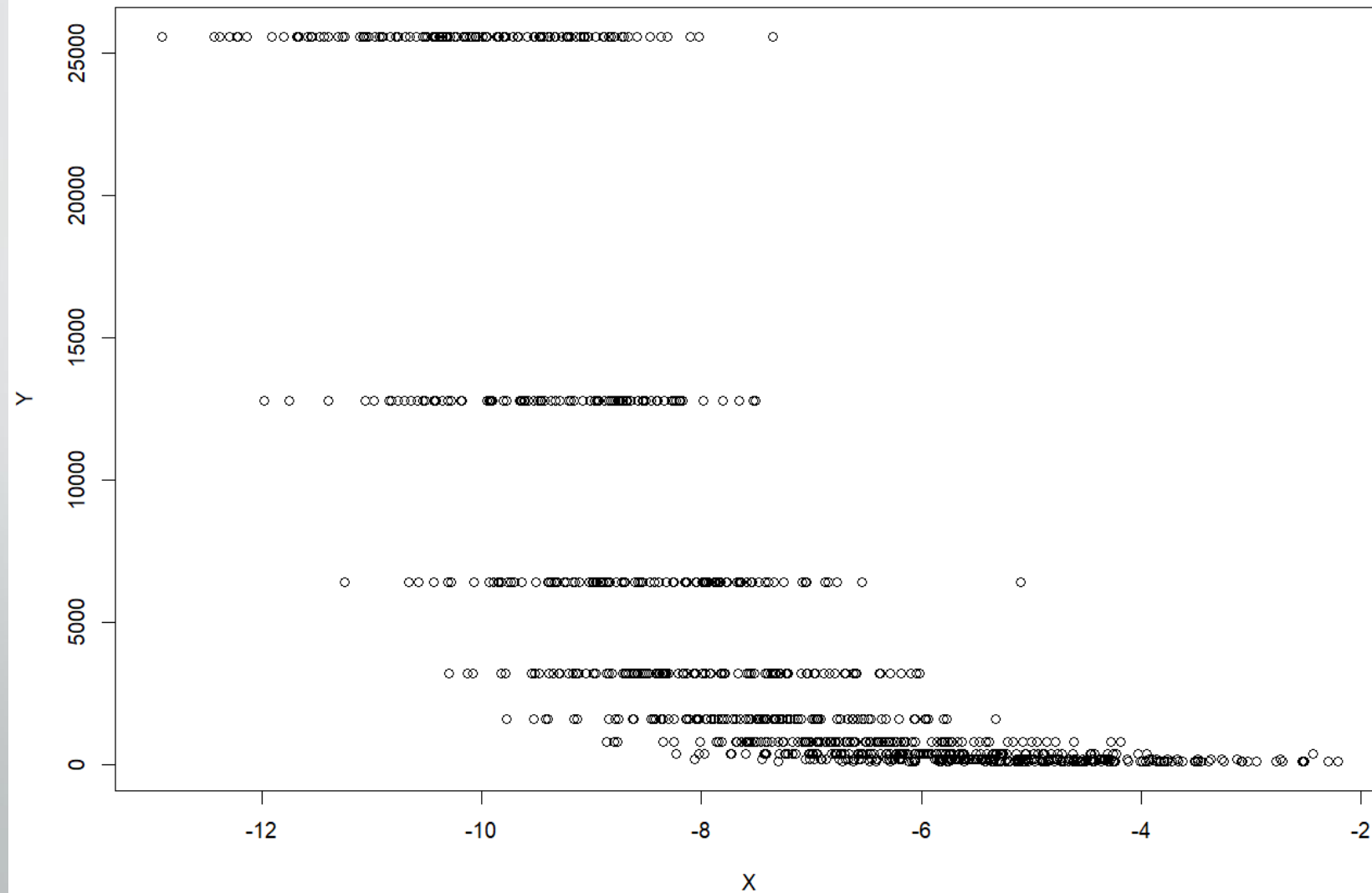
Seek a value of λ such that $Y_i^{(\lambda)} = X\beta_i + \epsilon_i$ with $\epsilon_i \sim N(0, \sigma^2)$

Common in practice: $\lambda \in [-2, 2]$

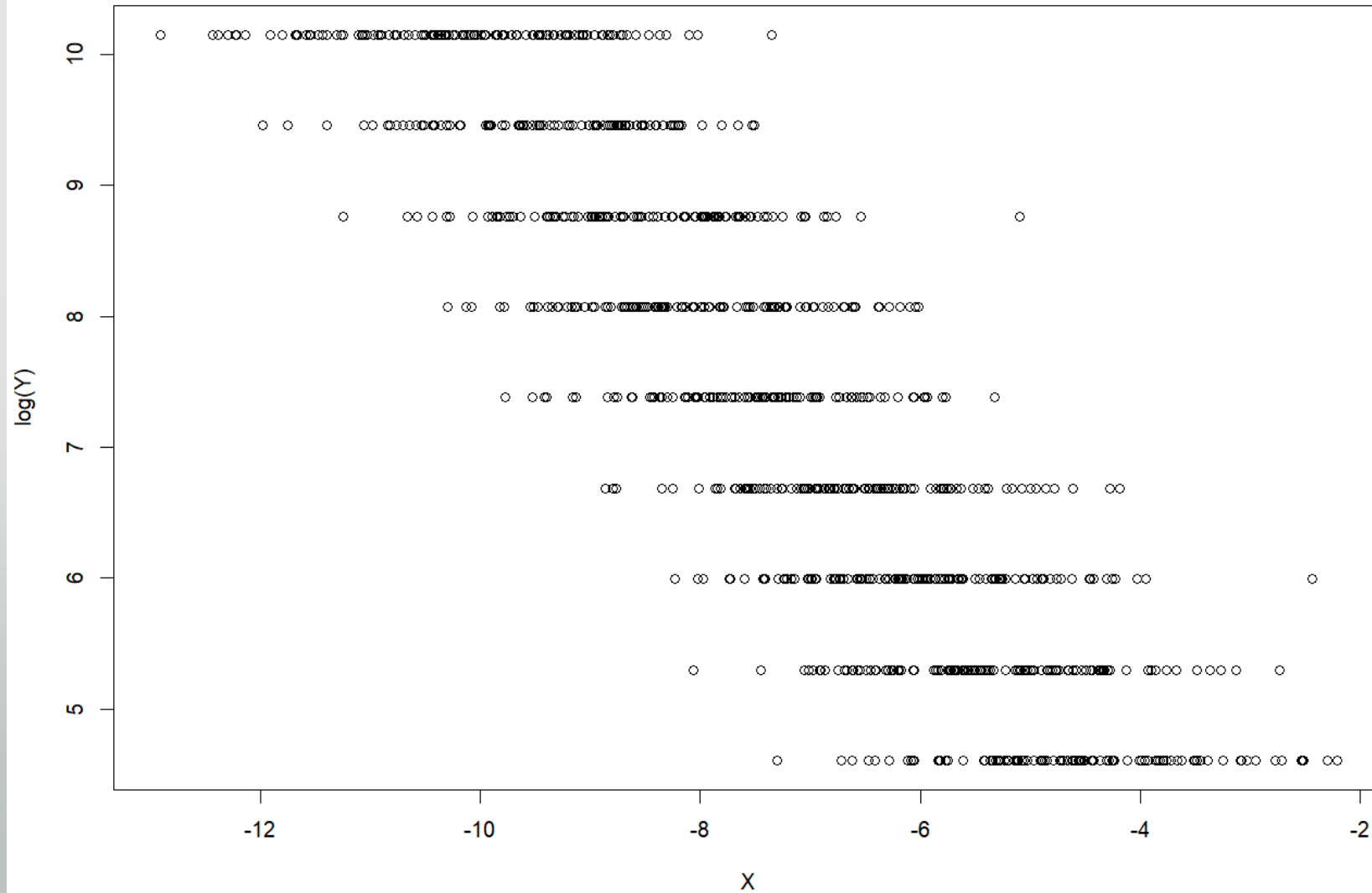
$$Y^{(\lambda)} = \begin{cases} \frac{Y^\lambda - 1}{\lambda} & \text{for } \lambda \neq 0 \\ \ln(Y) & \text{for } \lambda = 0 \end{cases}$$

$\lambda = 2$	Squared Transformation
$\lambda = 1$	No Transformation
$\lambda = 0.5$	Square Root Transformation
$\lambda = 0$	Log Transformation
$\lambda = -0.5$	Inverse Square Root Transformation
$\lambda = -1$	Inverse Transformation
$\lambda = -2$	Inverse Squared Transformation

Log transformation of Y



Log transformation of Y



Log transformation of Y

- Potential issues with log transformation
 - Values ≤ 0 will cause issues
- Common tweaks:
 - Can add small increment to 0 values in data
 - Can add constant to all values. But what constant?
- $y' = \log(y + \alpha) \rightarrow$ want to find α
- Can use the `logtrans()` function in the MASS package to find α

Log transformation of Y

- Potential issues with log transformation
 - Values ≤ 0 will cause issues
- Common tweaks:
 - Can add small increment to 0 values in data
 - Can add constant to all values. But what constant?
- $y' = \log(y + \alpha) \rightarrow$ want to find α
- Can use the `logtrans()` function in the MASS package to find α

Approach comes with some serious drawbacks as it can reduce interpretability, complicate assessment of goodness of fit, results in lost information, etc.

Interpretation of transformed variable

Common Box-Cox Transformations	
Lambda value (λ)	Transformed data (Y')
-3	$Y^{-3} = 1/Y^3$
-2	$Y^{-2} = 1/Y^2$
-1	$Y^{-1} = 1/Y^1$
-0.5	$Y^{-0.5} = 1/(\sqrt{Y})$
0	$\log(Y)^{**}$
0.5	$Y^{0.5} = \sqrt{Y}$
1	$Y^1 = Y$
2	Y^2
3	Y^3

<https://www.statisticshowto.com/probability-and-statistics/normal-distributions/box-cox-transformation/>

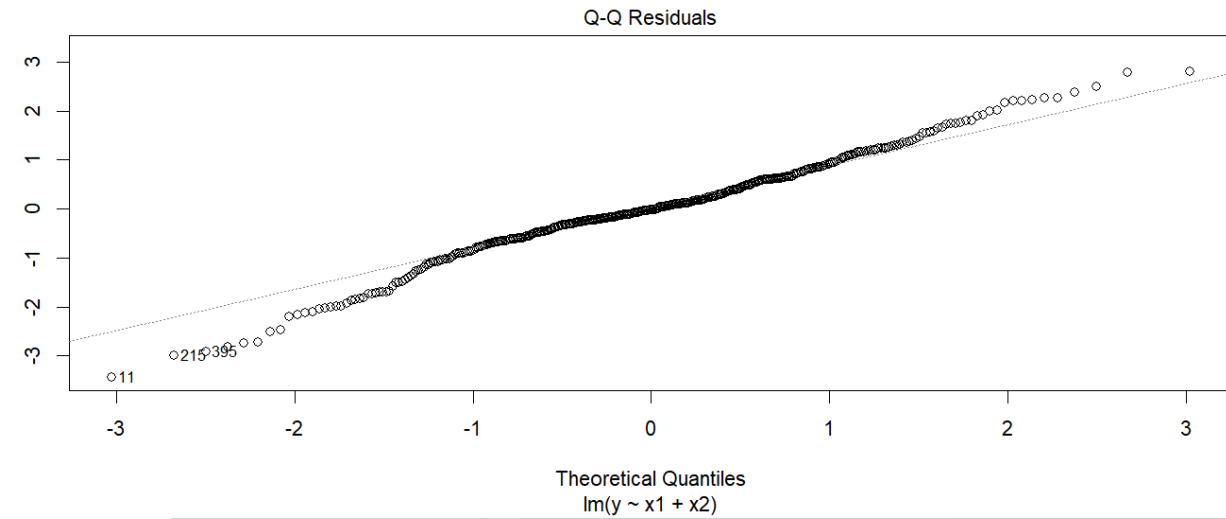
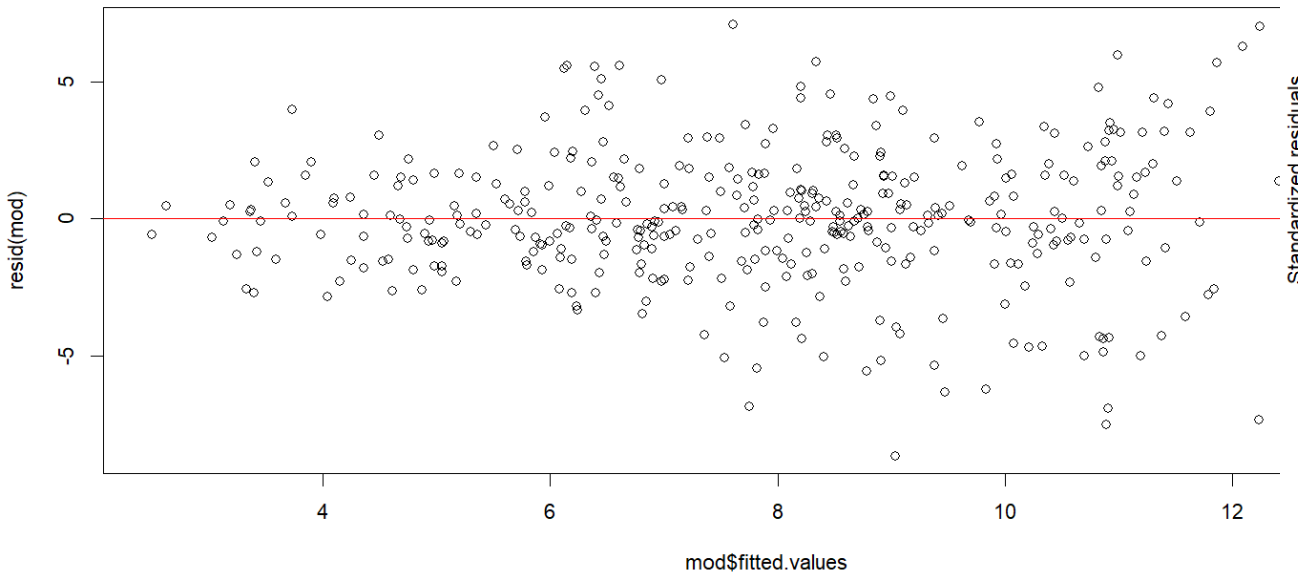
Example

- Let's start with some simulated data
- We'll first fit a linear regression model using OLS:
 - `mod <- lm(y ~ x1 + x2, dat)`

```
5 library(mgcv)
6 library(MASS)
7 set.seed(123)
8 dat = gamSim(
9   1,
10  n = 400,
11  dist = "normal",
12  scale = 1,
13  verbose = FALSE
14 )
```

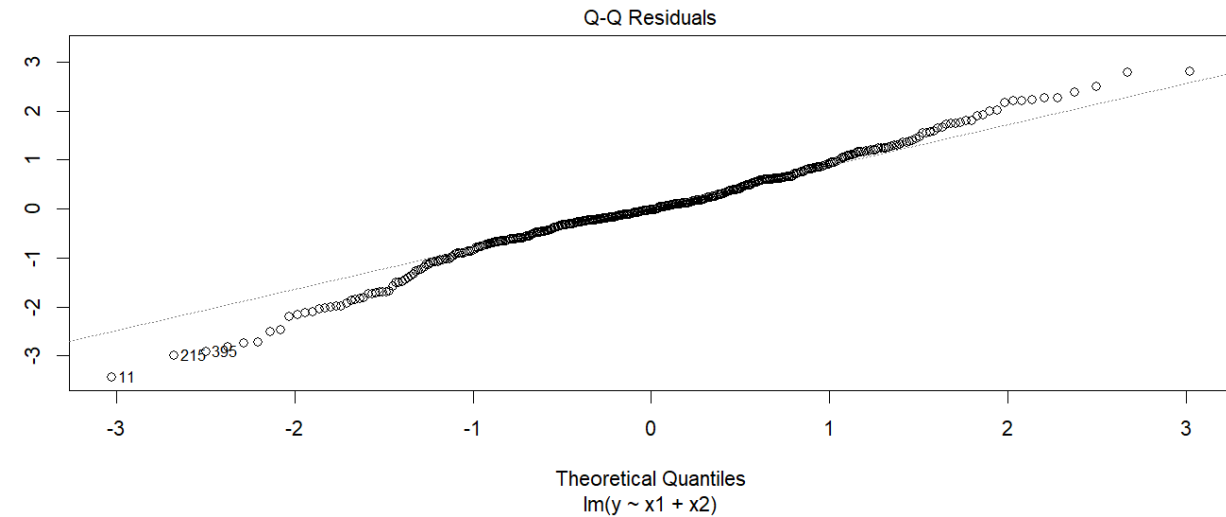
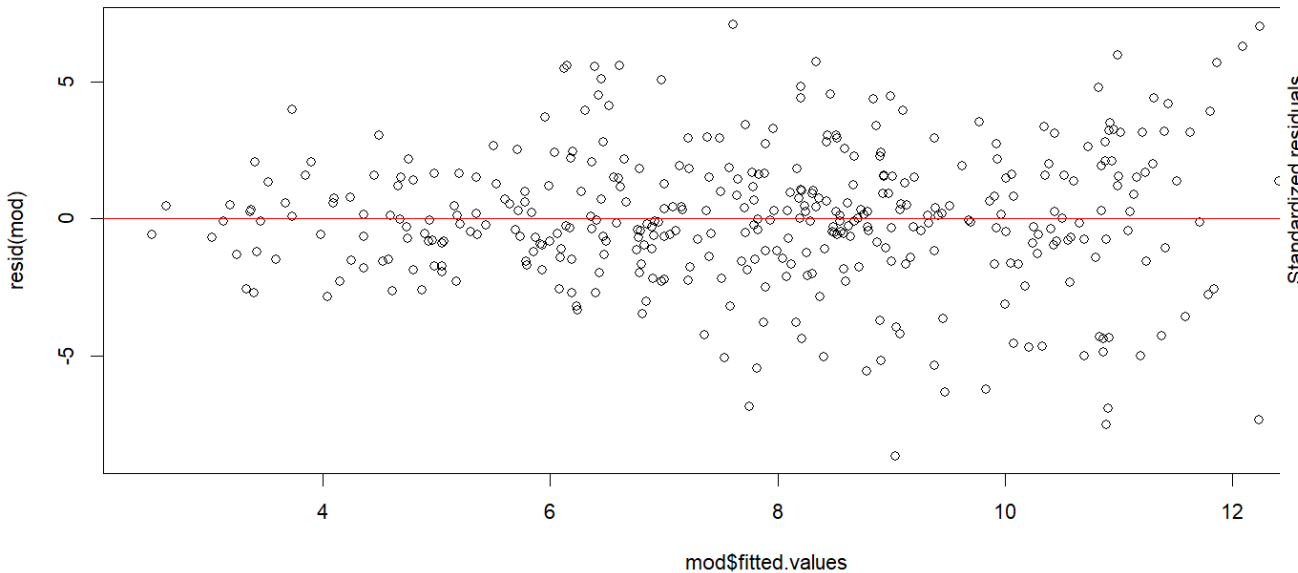
Example

- Now let's check our OLS assumptions, particularly homoscedasticity and error normality.
- What do you see?



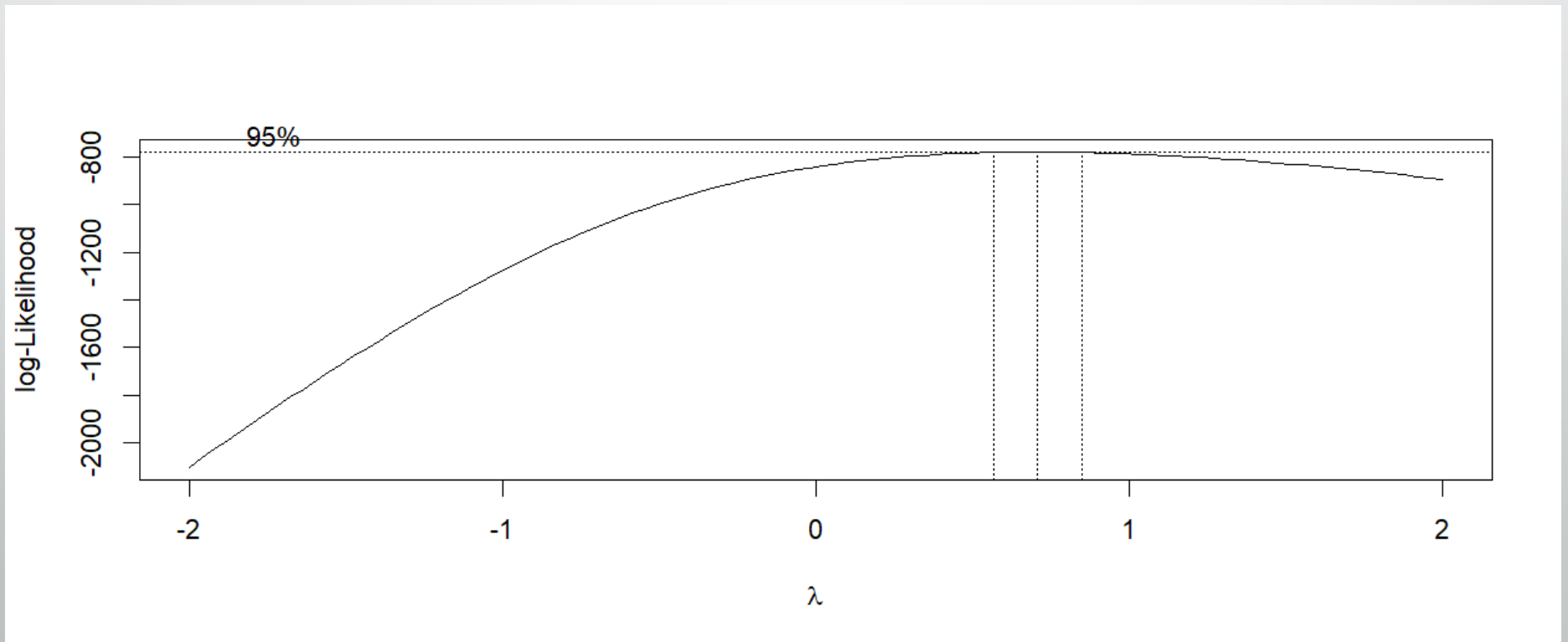
Example

- Now let's check our OLS assumptions, particularly homoscedasticity and error normality.
- What do you see? → Clear funnel shape and some issues with the tails.



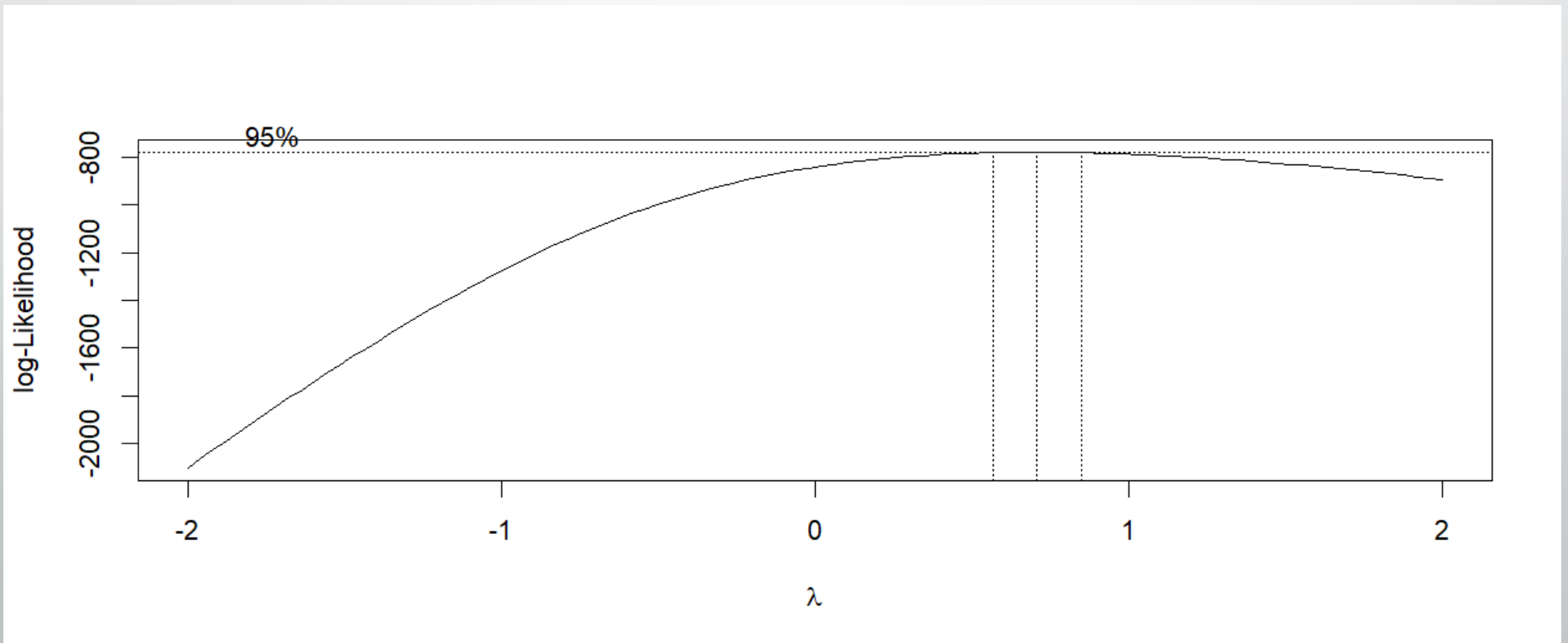
Example

- We can use the `boxcox()` function to see if any transformations might make sense.
- In between 0.5 (square root transformation) and 1 (no transformation).
- Let's try a square root transformation.



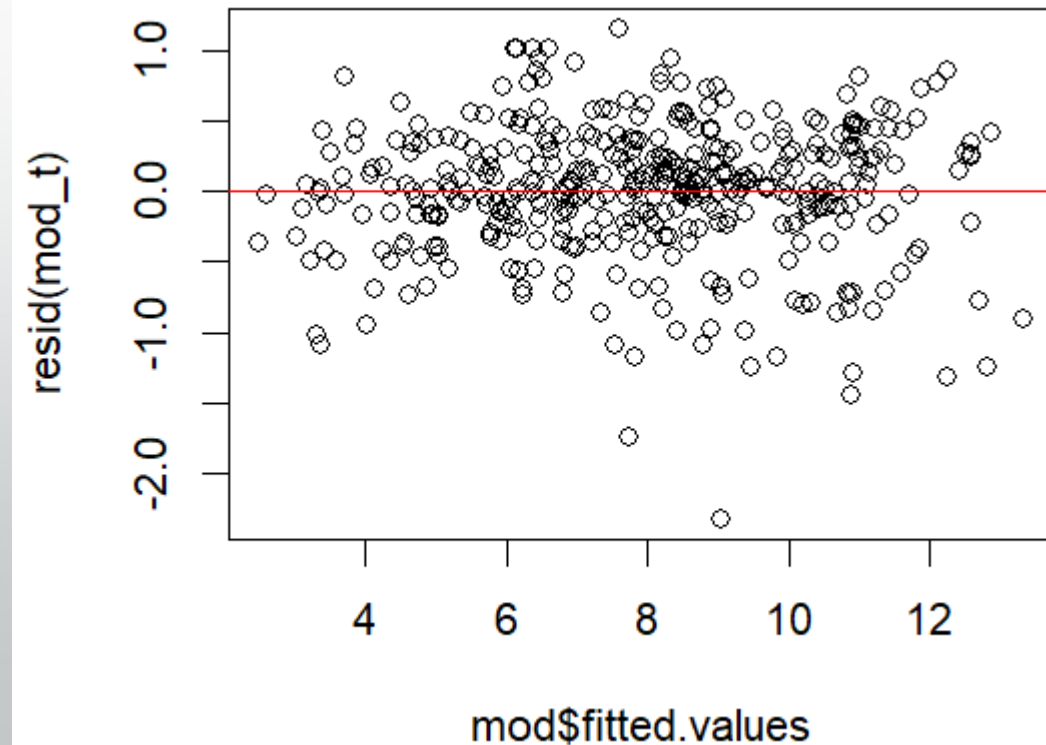
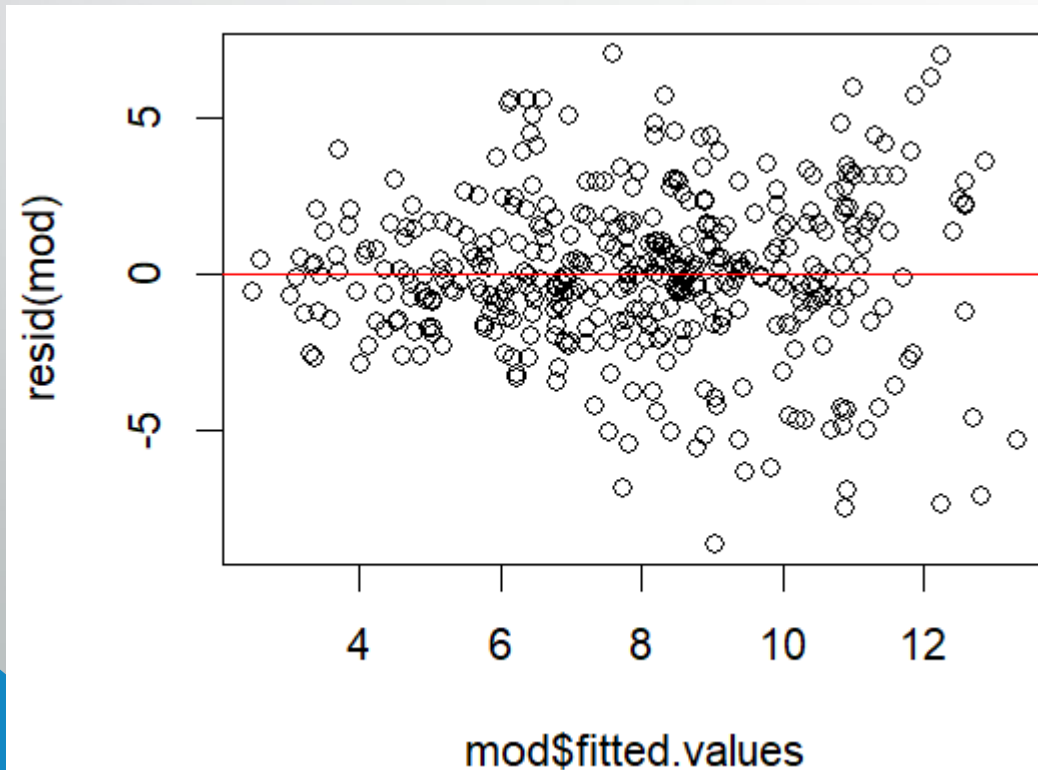
Example

- We can use the `boxcox()` function to see if any transformations might make sense.
- In between 0.5 (square root transformation) and 1 (no transformation).
- Let's try a square root transformation.
 - `mod_t <- lm(sqrt(y) ~ x1 + x2, dat)`



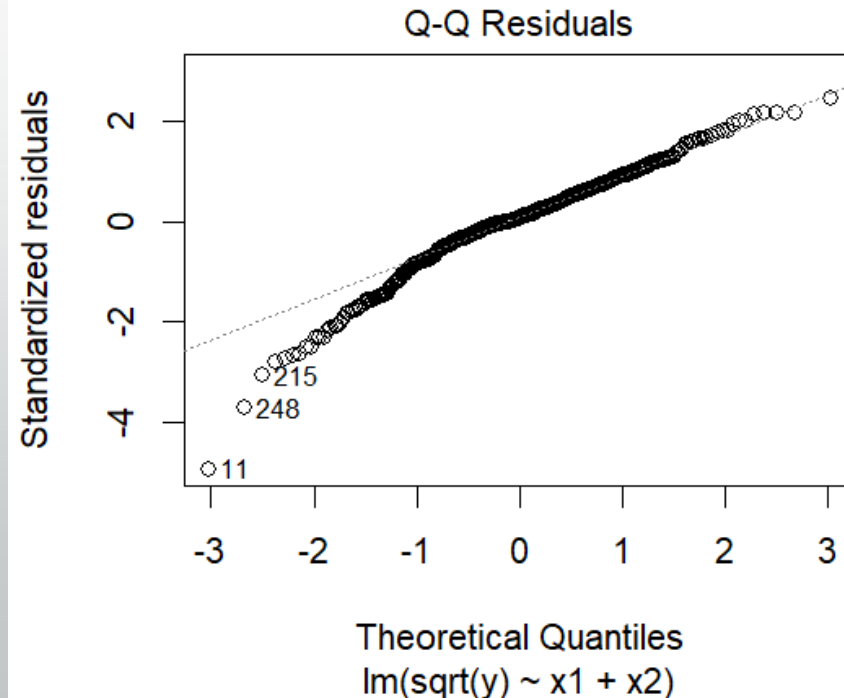
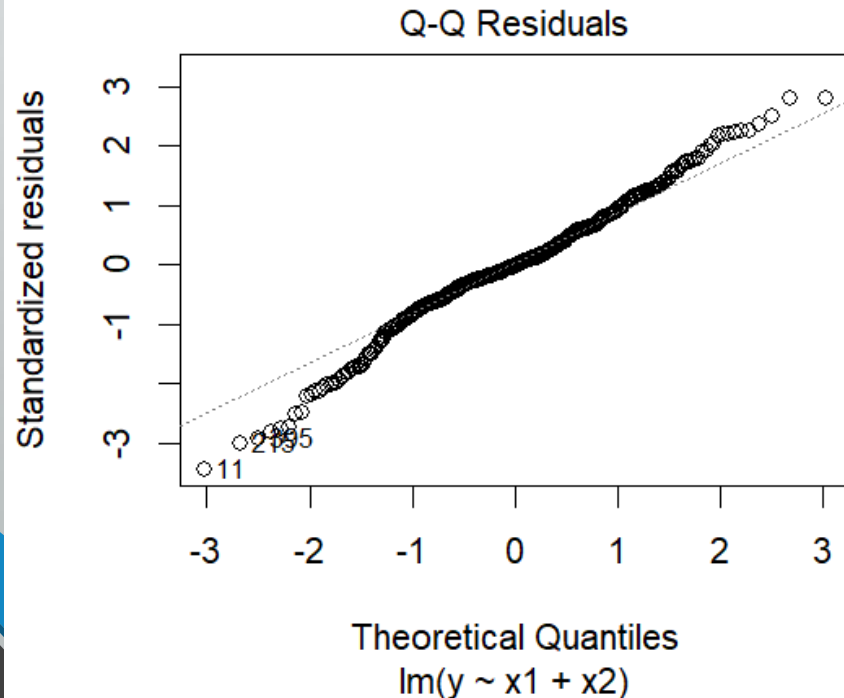
Example

- We can use the `boxcox()` function to see if any transformations might make sense.
- In between 0.5 (square root transformation) and 1 (no transformation).
- Let's try a square root transformation.
 - `mod_t <- lm(sqrt(y) ~ x1 + x2, dat)`



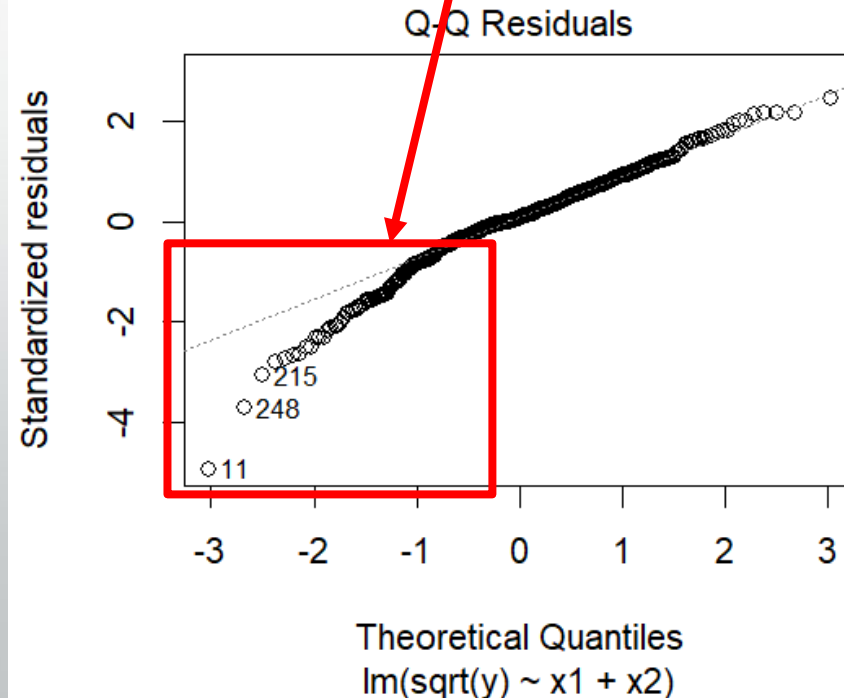
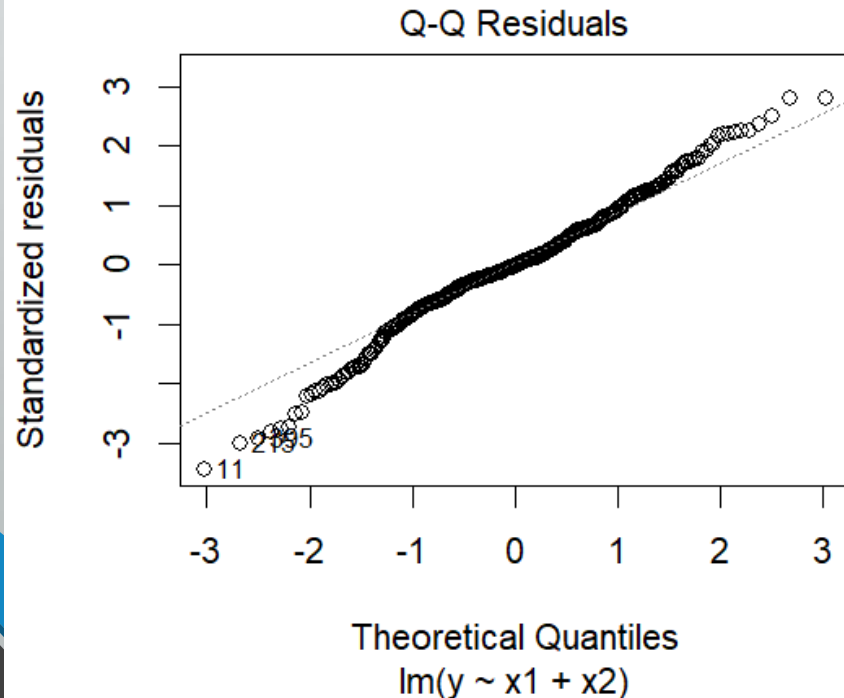
Example

- We can use the `boxcox()` function to see if any transformations might make sense.
- In between 0.5 (square root transformation) and 1 (no transformation).
- Let's try a square root transformation.
 - `mod_t <- lm(sqrt(y) ~ x1 + x2, dat)`



Example

- We can use the `boxcox()` function to see if any transformations might make sense.
- In between 0.5 (square root transformation) and 1 (no transformation).
- Let's try a square root transformation.
 - `mod_t <- lm(sqrt(y) ~ x1 + x2, dat)`

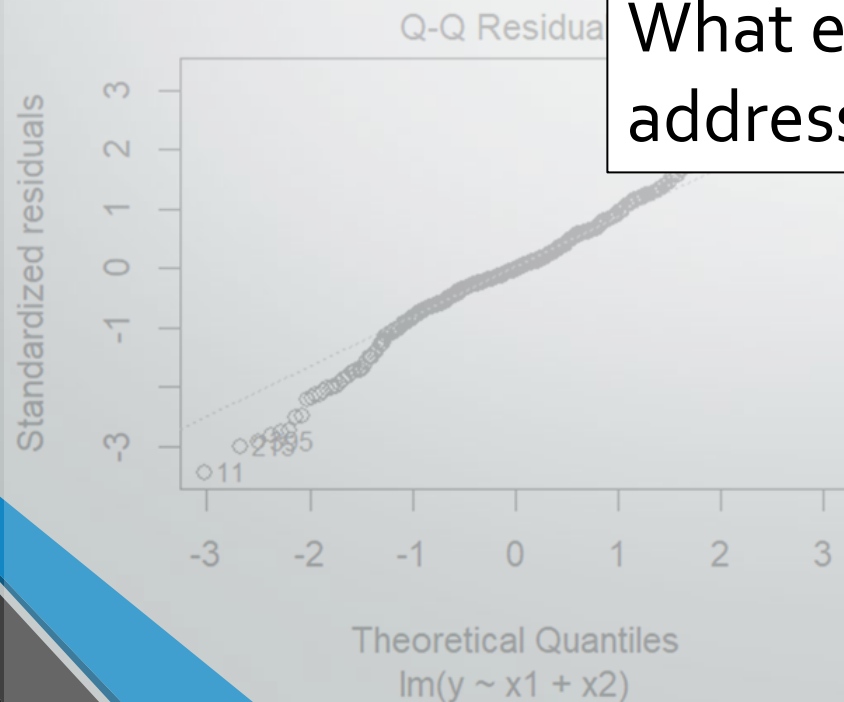


Example

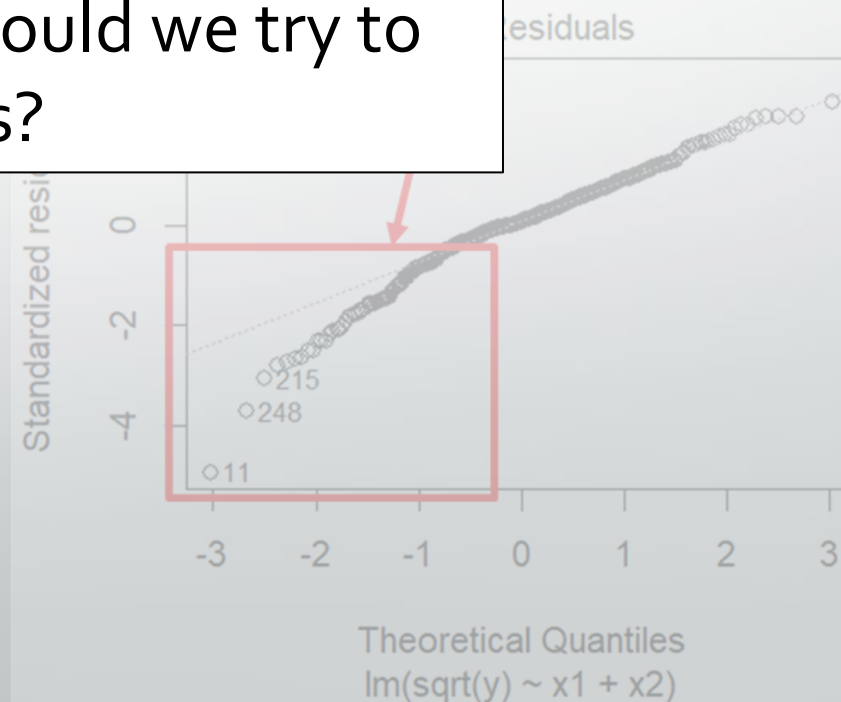
- We can use the `boxcox()` function to see if any transformations might make sense.
- In between 0.5 (square root transformation) and 1 (no transformation).
- Let's try a square root transformation.

- `mod_t <- lm(sqrt(y) ~ x1 + x2, dat)`

Helped a bit, but still issues with the lower tail.



What else could we try to address this?

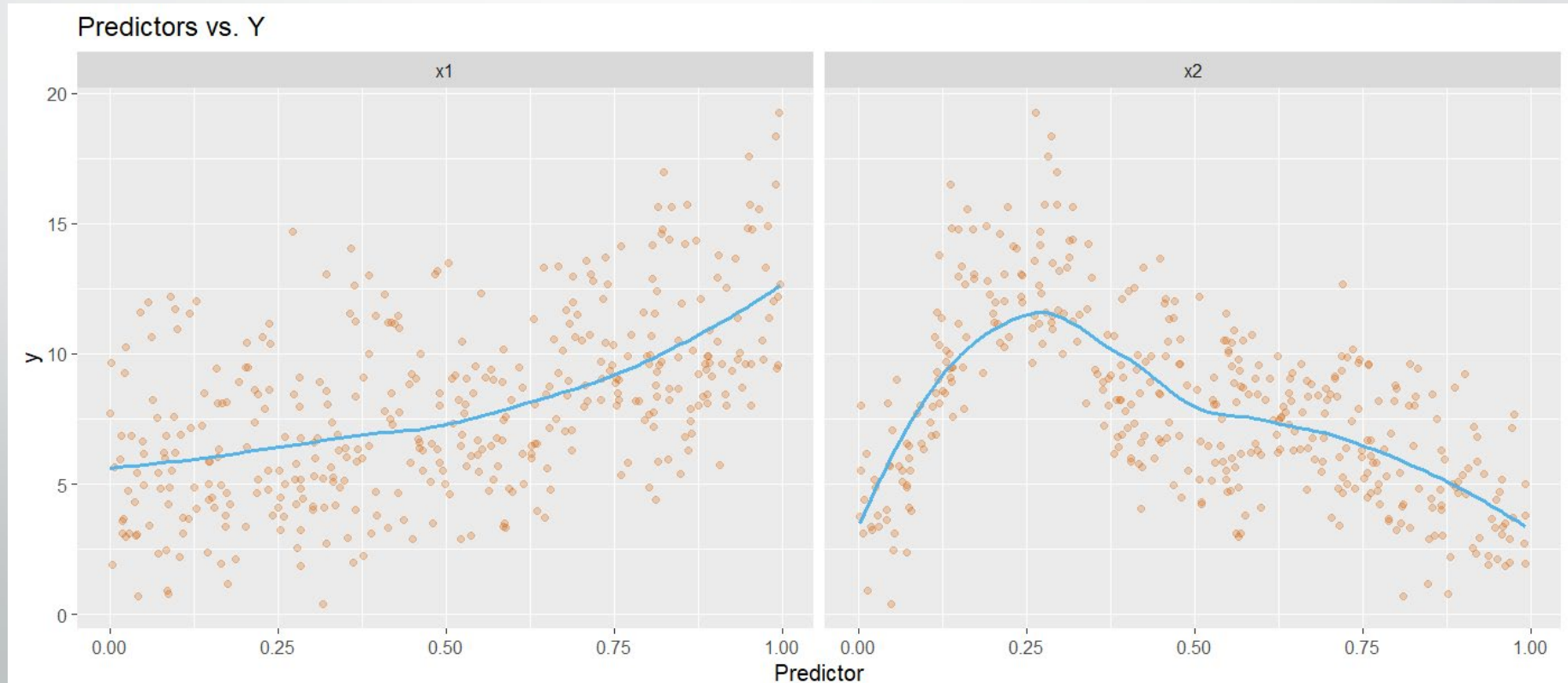




Polynomial regression

Polynomial Regression

- Polynomial regression models can account for non-linear relationships by including different order polynomials (e.g., quadratic x^2 , cubic x^3 , etc.) as predictors.

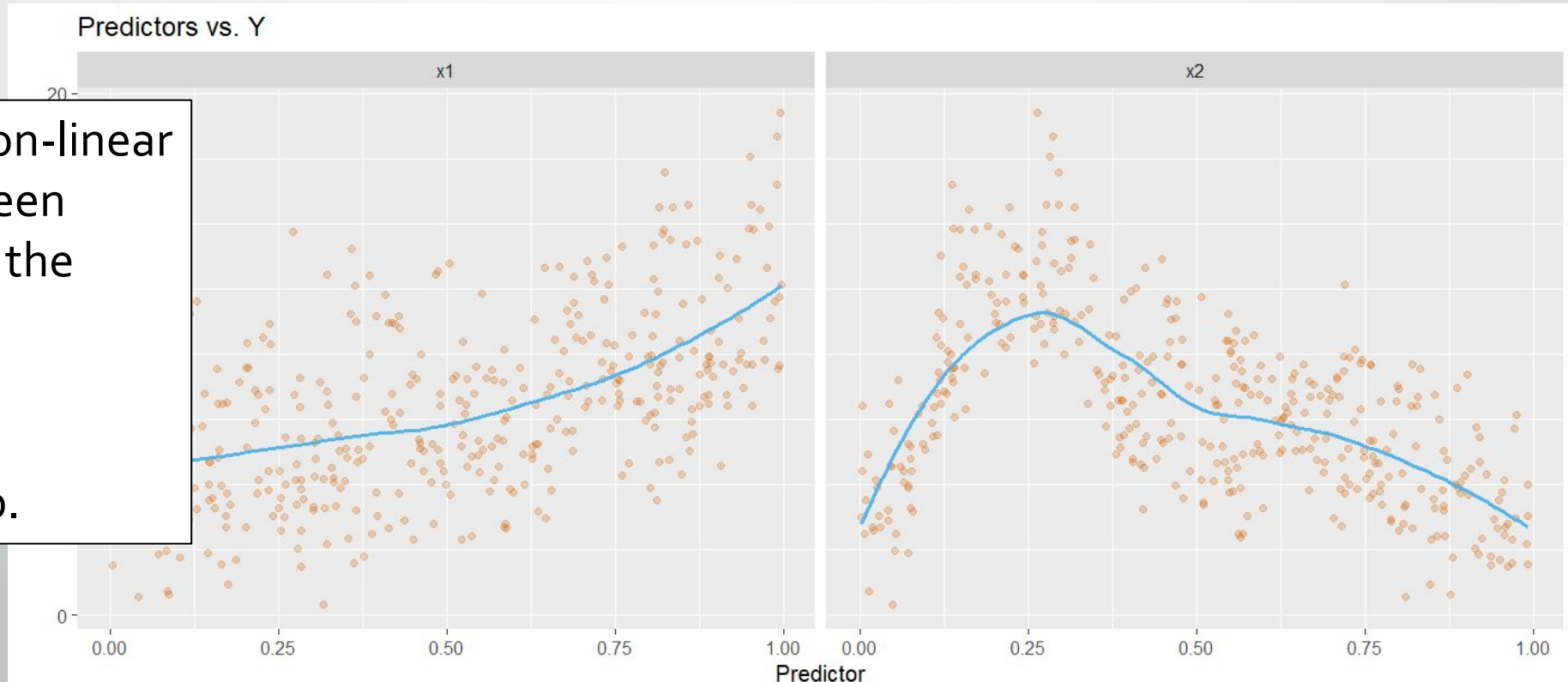


Polynomial Regression

- Polynomial regression models can account for non-linear relationships by including different order polynomials (e.g., quadratic x^2 , cubic x^3 , etc.) as predictors.

We can see clear non-linear relationships between each predictor and the outcome.

Maybe polynomial regression can help.



Polynomial Regression

Original model: `lm(y ~ x1 + x2, dat)`

Transformed model: `lm(sqrt(y) ~ x1 + x2, dat)`

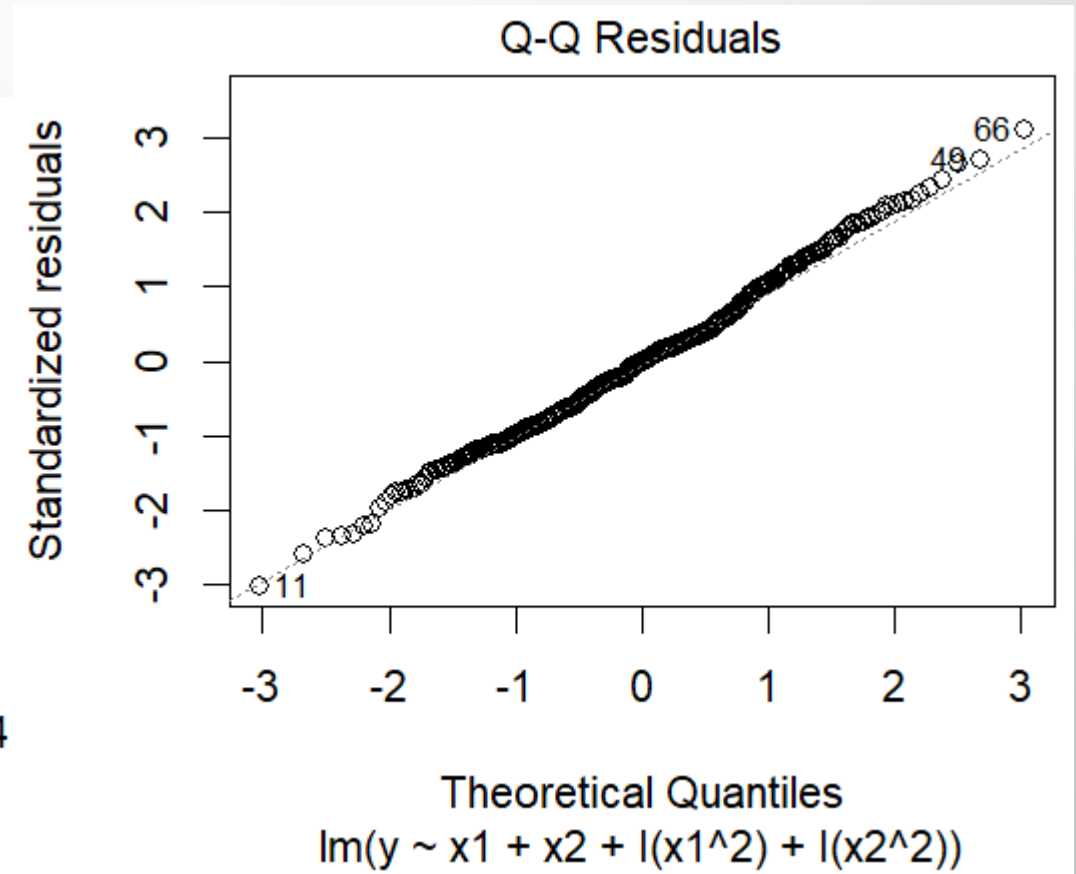
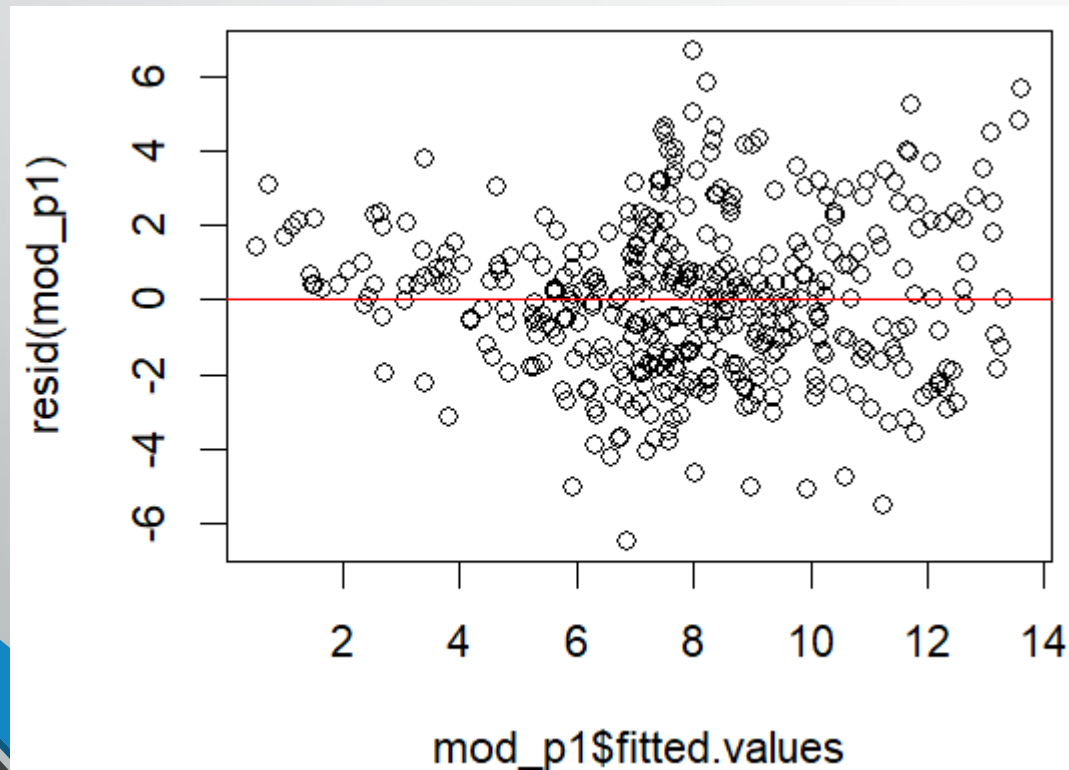
- Let's try fitting 2 polynomial regression models:

Polynomial model 1: `lm(y ~ x1 + x2 + I(x1^2) + I(x2^2), dat)`

Polynomial model 2: `lm(y ~ x1 + x2 + I(x1^2) + I(x2^2) + I(x2^3), dat)`

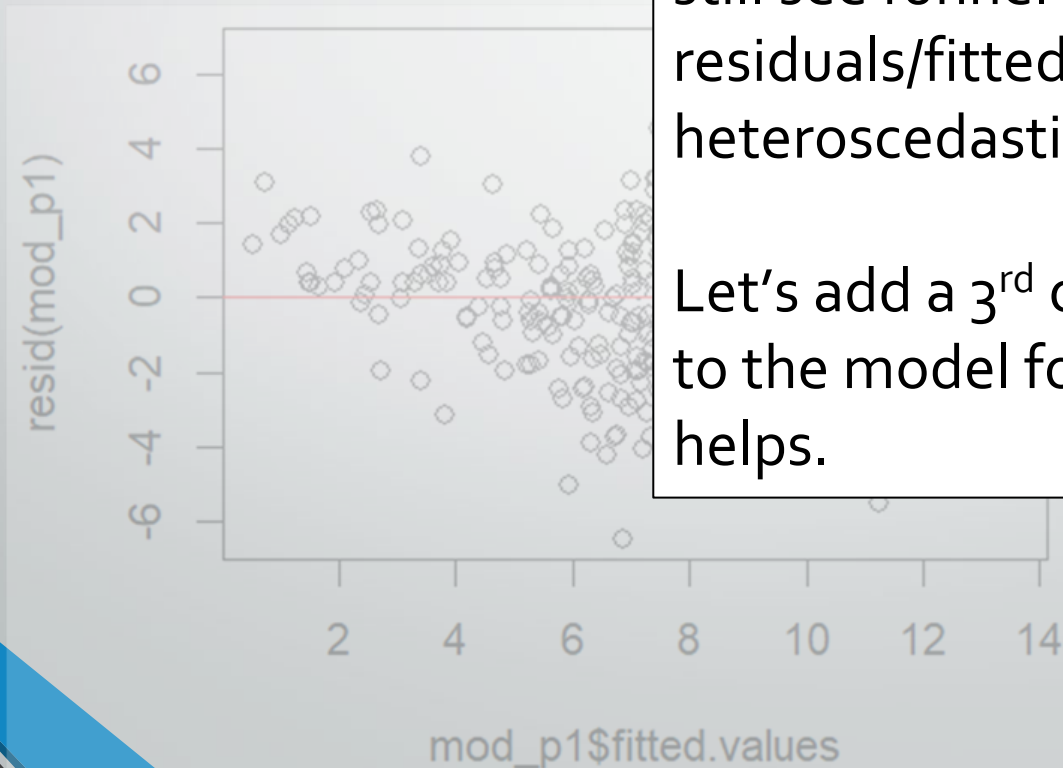
Polynomial Regression

Polynomial model 1: $\text{lm}(y \sim x1 + x2 + I(x1^2) + I(x2^2), \text{dat})$



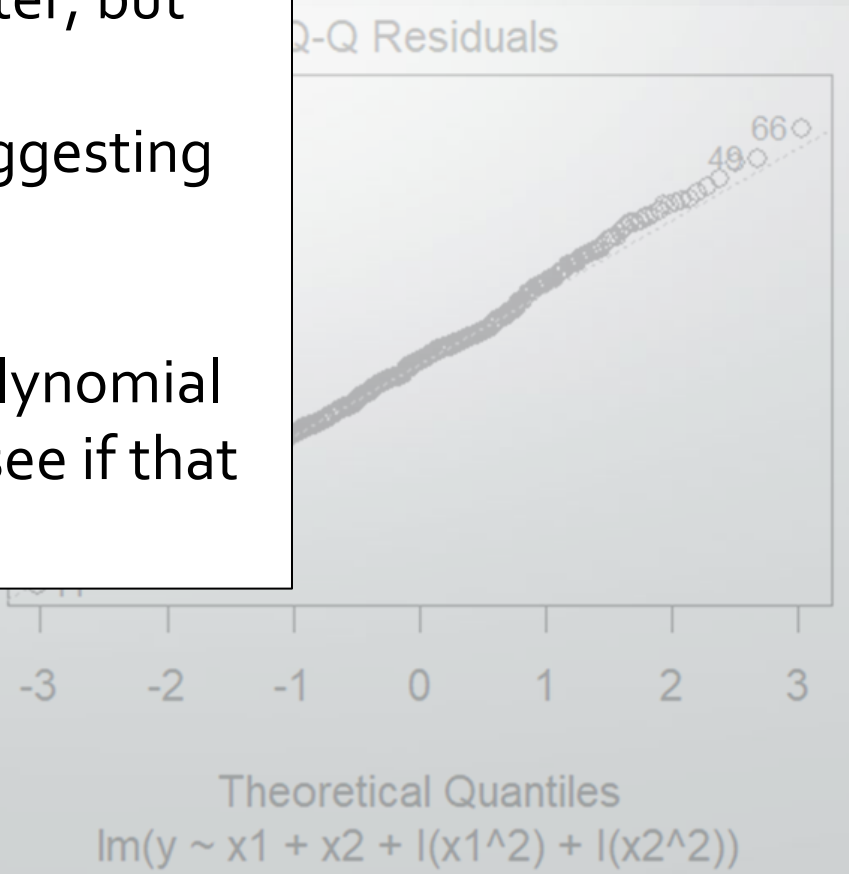
Polynomial Regression

Polynomial model 1: `lm(y ~ x1 + x2 + I(x1^2) + I(x2^2), dat)`



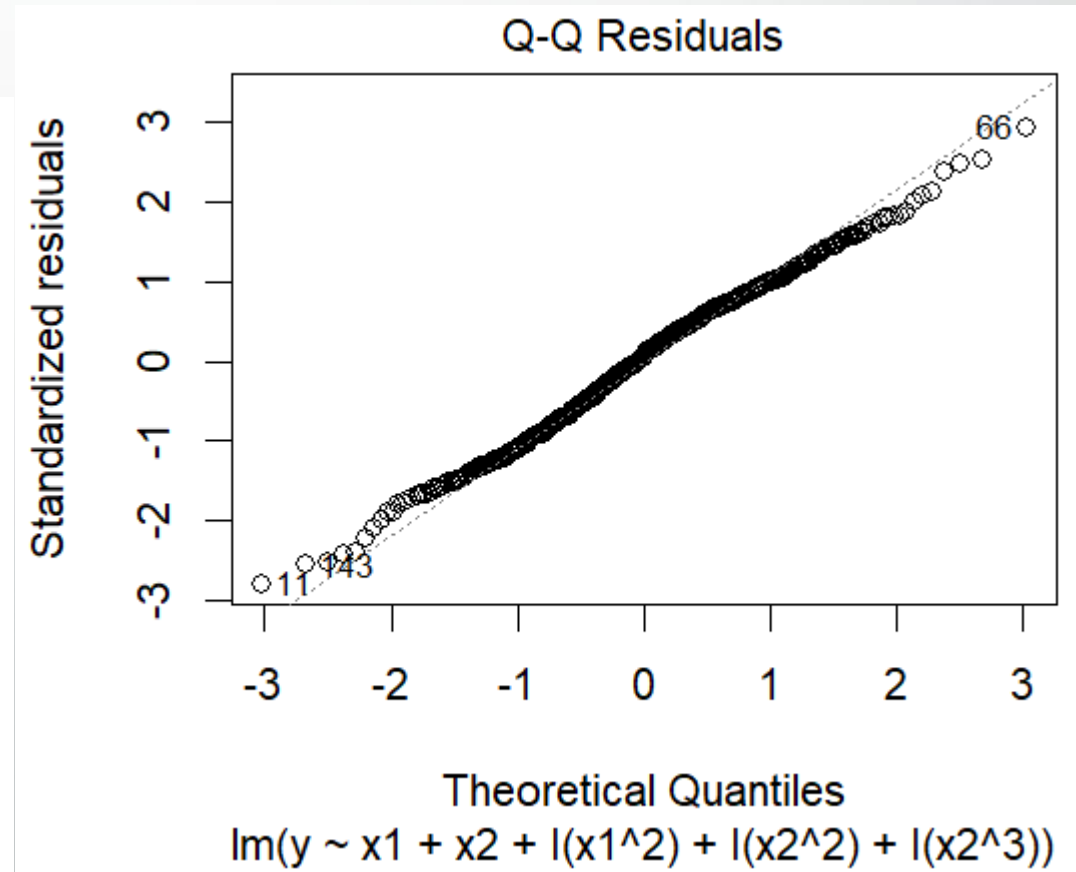
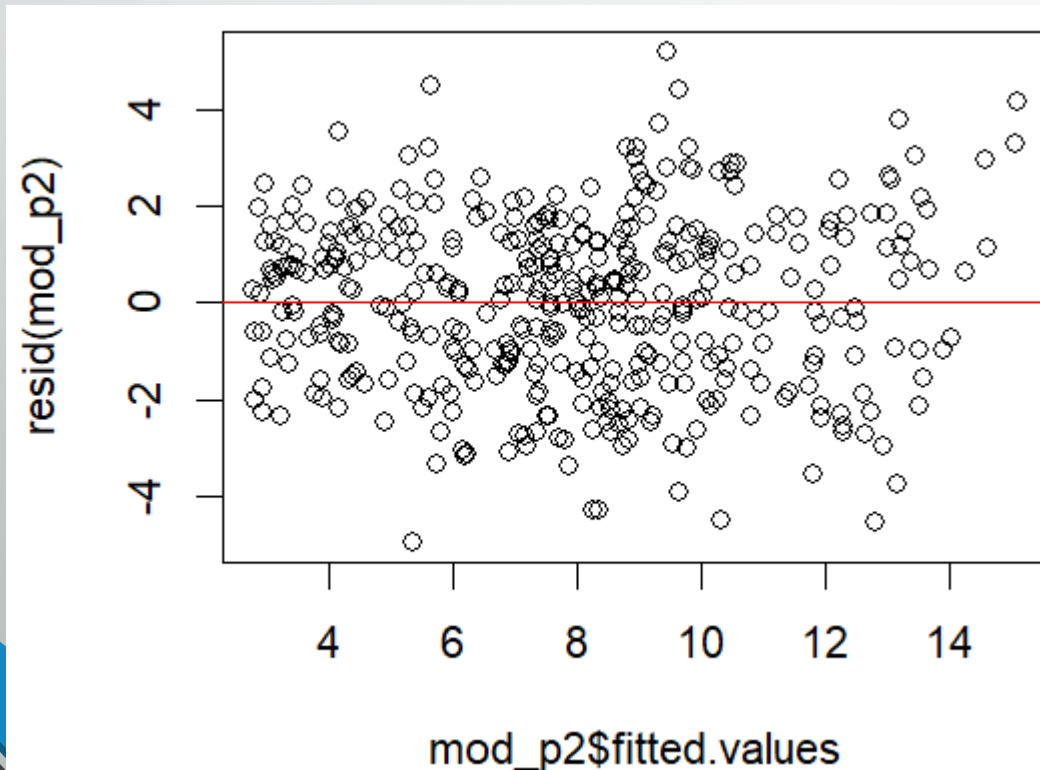
QQ plot looks a lot better, but still see funnel shape in residuals/fitted plot suggesting heteroscedasticity.

Let's add a 3rd order polynomial to the model for x2 to see if that helps.



Polynomial Regression

Polynomial model 2: $\text{lm}(y \sim x1 + x2 + I(x1^2) + I(x2^2) + I(x2^3), \text{dat})$

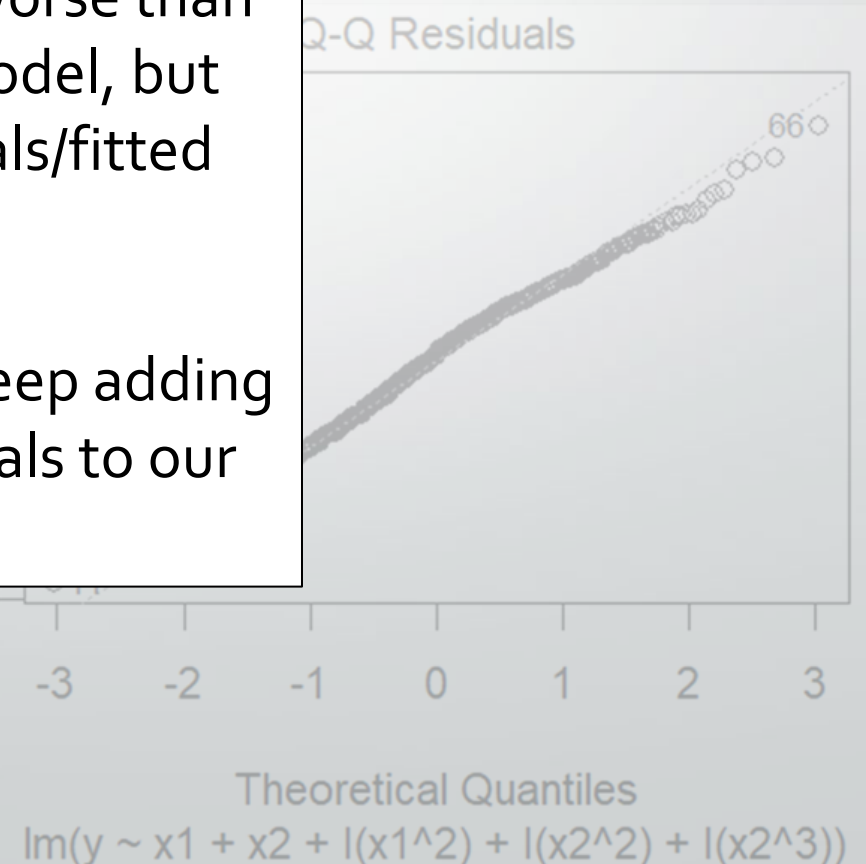
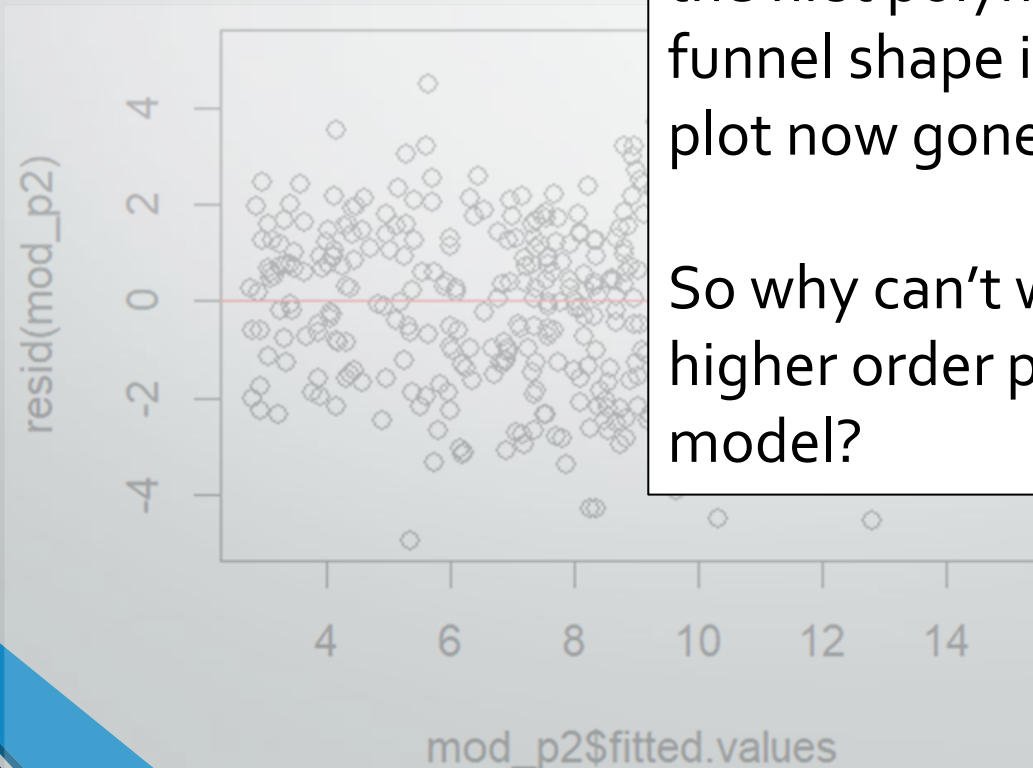


Polynomial Regression

Polynomial model 2: `lm(y ~ x1 + x2 + I(x1^2) + I(x2^2) + I(x2^3), dat)`

QQ plot looks a little worse than the first polynomial model, but funnel shape in residuals/fitted plot now gone.

So why can't we just keep adding higher order polynomials to our model?



Polynomial Regression

- Rarely will true exposure:outcome relationship actually be a quadratic or cubic relationship.
 - Can be a useful approximation, but very prone to overfitting.
- These limitations become particularly clear when we have more complex non-linear relationships.

Polynomial Regression – Example 2

- Let's look at an example with a more complex relationship using some synthetic data.
- We'll then fit multiple polynomial regression models with increasingly higher-order terms

```
set.seed(123)

x = runif(500)
mu = sin(2 * (4 * x - 2)) + 2 * exp(-(16 ^ 2) * ((x - .5) ^ 2))
y = rnorm(500, mu, .3)
d = data.frame(x, y)
```

Polynomial Regression – Example 2

- Let's look at the data.
- We'll then fit a polynomial regression model with higher-order terms

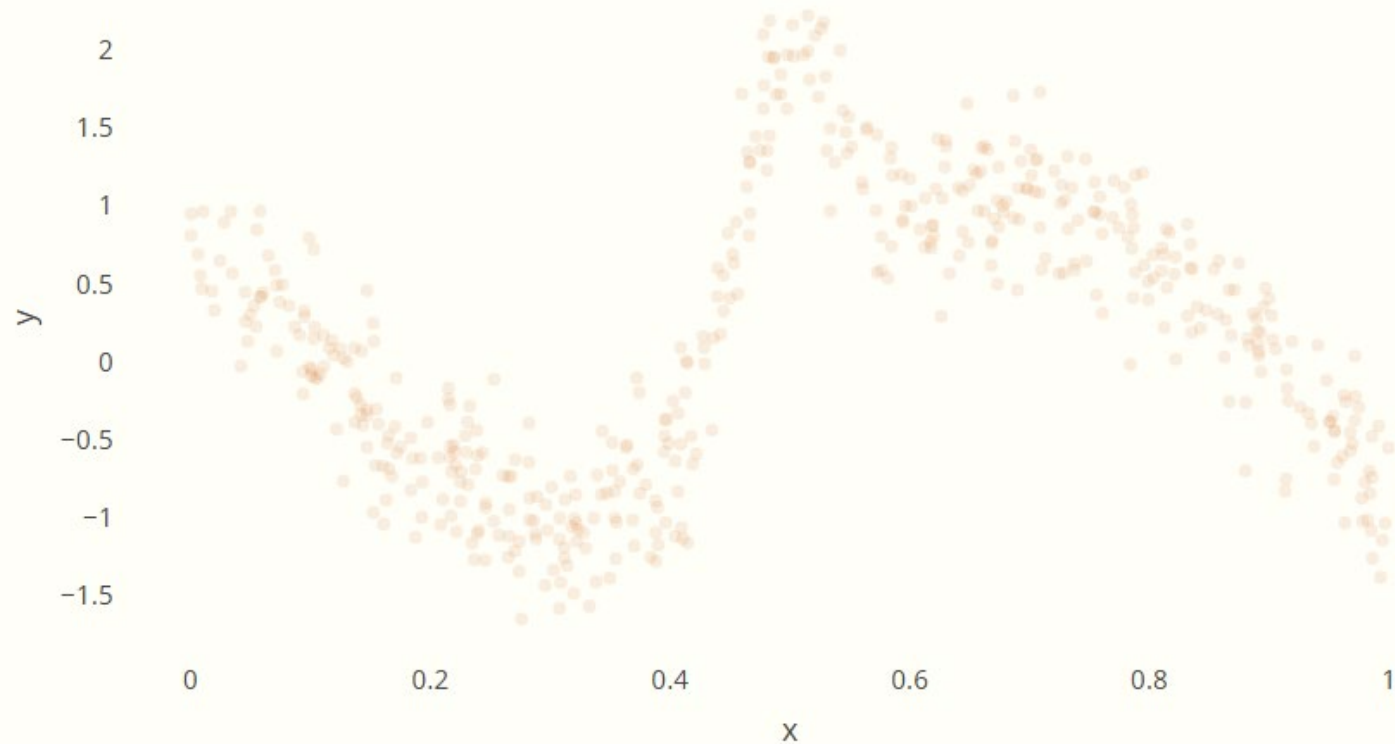
```
set.seed(12345)
```

```
x = runif(1000)
```

```
mu = sin(2 * pi * x)
```

```
y = rnorm(1000, mu, 0.5)
```

```
d = data.frame(x, y)
```

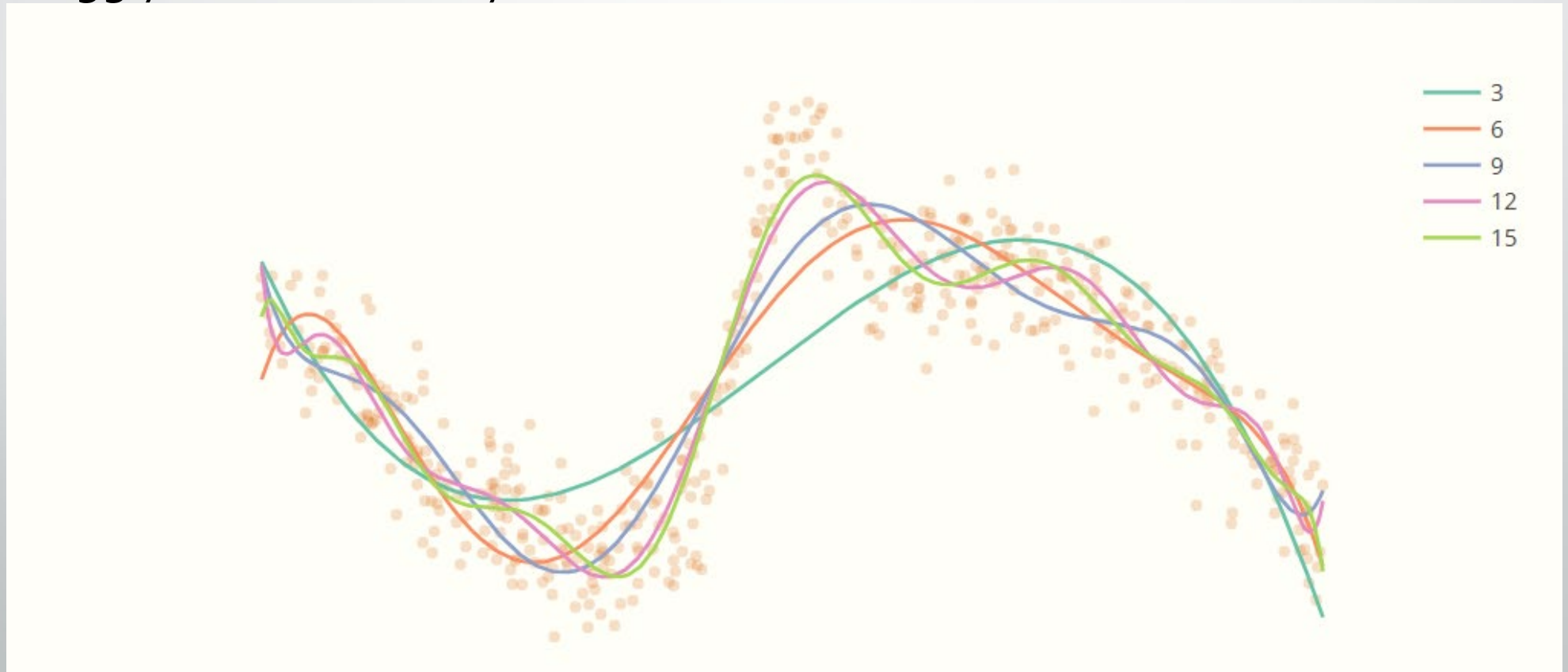


the synthetic

ly higher-order

Polynomial Regression – Example 2

- Higher order polynomials still fit the data poorly in some areas and is frequently more “wiggly” than necessary.



Generalized Additive Models (GAMs)

- Generalized additive models with smoothed predictors offer a solution

Generalized additive model:

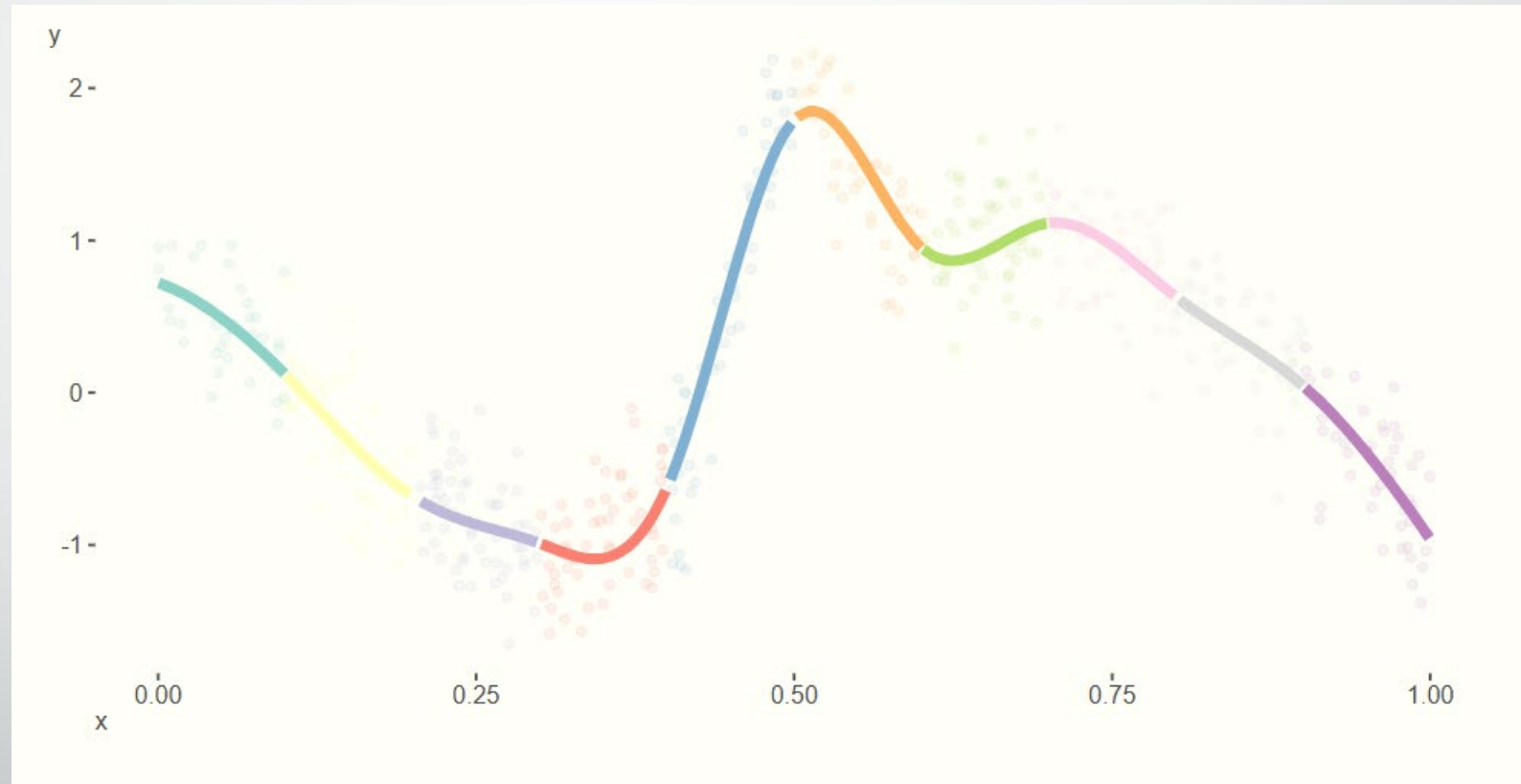
$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \epsilon$$

Linear regression model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$$

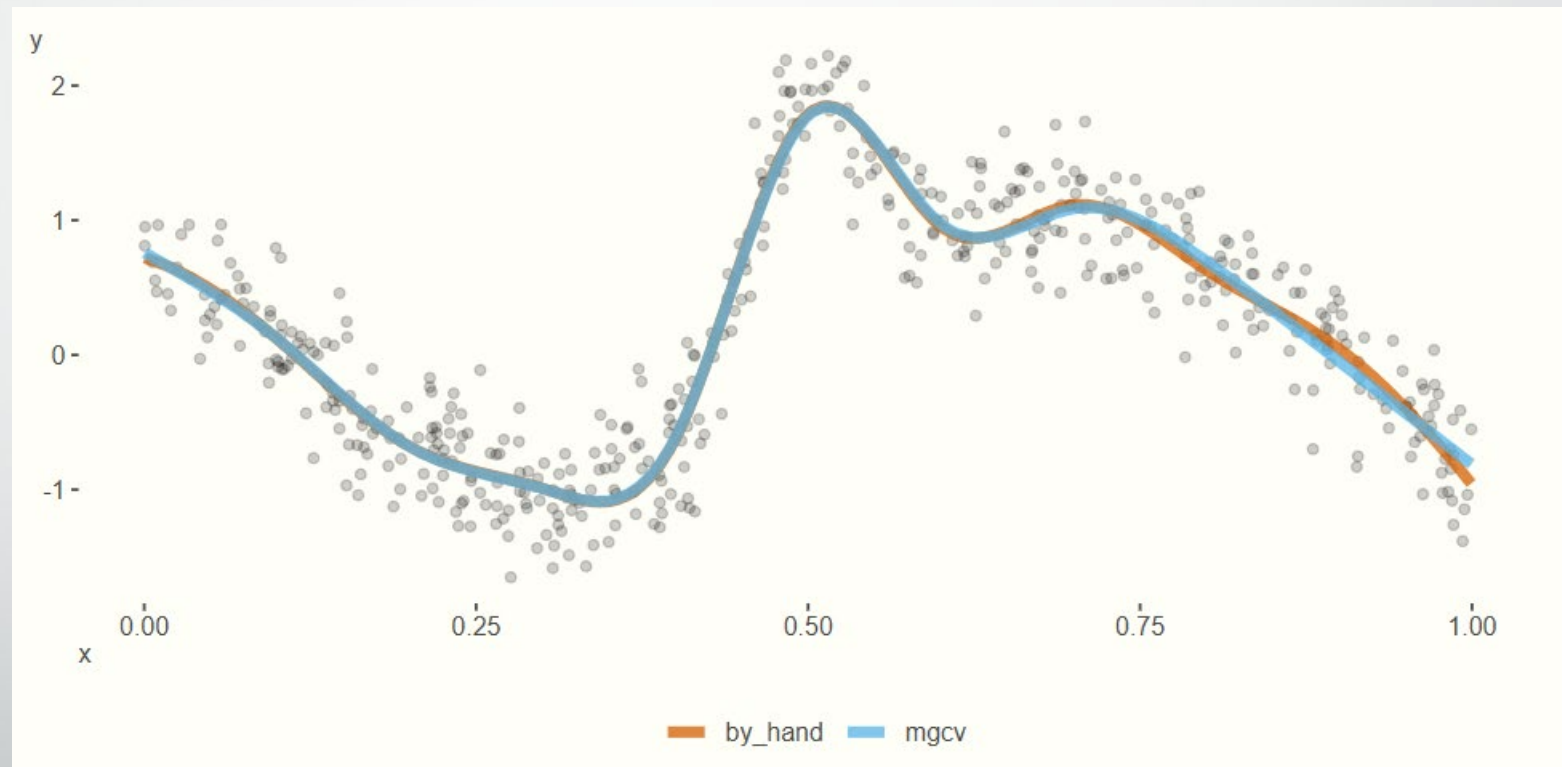
Generalized Additive Models (GAMs)

Essentially can be thought of as breaking the model into different sections with each section fit using a function.



Generalized Additive Models (GAMs)

Putting it all together we get something like this.





Model Selection



Background

Why model selection?

- Model parsimony
 - Occam's razor: "among several plausible explanations for a phenomenon, the simplest is often the best"
 - Can reduce noise (i.e., wasted degrees of freedom)
 - Can reduce data collection resources
- Depends on what you're trying to do with the model
 - Explain a given phenomenon
 - Predict future observations

Model Selection Methods

- Technical procedures
 - Model comparison: General F -test (Class #5)
 - Predictor significance
 - Criterion-based: Akaike Information Criteria (AIC), Bayesian Information Criterion (BIC), Mallows's C_p , Adjusted R^2 , many more
 - Forward, Backward, Stepwise selection
 - These are all just techniques – consider as a reference
- Theoretical procedures
 - Explaining an outcome variable
 - Effect of a particular predictor
 - ****If there are important predictors, do not drop them merely based on the results of statistical techniques****
 - **Consult subject matter of the analysis** first and then consider carrying out mechanical/statistical procedures

General F -Test

- The more predictors to a model, the better the model explains the outcome, unless $Y \perp\!\!\!\perp X$
- We want parsimony in our model
- How do we know which model is better?

Consider a full model with k predictors (including an intercept) and a reduced model with p predictors

$$\begin{aligned} F &= \frac{(RSS_{Reduced} - RSS_{Full}) / (df_{Reduced} - df_{Full})}{RSS_{Full} / df_{Full}} \\ &= \frac{(RSS_{Reduced} - RSS_{Full}) / (k - p)}{RSS_{Full} / (n - k)} \sim F_{n-k}^{k-p} \\ &\rightarrow \text{Test } H_0 : RSS_{Reduced} = RSS_{Full} \end{aligned}$$



Criterion-Based Procedures

Likelihood Function

- Likelihood function: $\mathcal{L}(\theta|y) = P(Y = y|\theta) = f(y|\theta)$
 - θ : parameters; y : observed data; f : probability density function
 - $\mathcal{L}(\theta|y) = \prod_i f(y_i|\theta)$
- Log-likelihood function: $\ell(\theta|y) = \log(\mathcal{L}(\theta|y))$
 - $\ell(\theta|y) = \log \prod_i (f(y_i|\theta)) = \sum_{i=1}^n \log(f(y_i|\theta)) = \sum_i \ell(\theta|y_i)$
 - More desirable properties than $\mathcal{L}(\theta|y)$ (e.g., concave [remember Box-Cox], calculation

Maximum Likelihood Estimation (MLE)

- Find the values of the model parameters that maximize the likelihood function over the parameter space (i.e., the likelihood that the model produces the observed data)
- Log-likelihood is used in this process by solving its derivative with respect to θ set to 0
- Model with high or low log-likelihood?

Likelihood Function

- Likelihood function: $\mathcal{L}(\theta|y) = P(Y = y|\theta) = f(y|\theta)$
 - θ : parameters; y : observed data; f : probability density function
 - $\mathcal{L}(\theta|y) = \prod_i f(y_i|\theta)$
- Log-likelihood function: $\ell(\theta|y) = \log(\mathcal{L}(\theta|y))$
 - $\ell(\theta|y) = \log \prod_i (f(y_i|\theta)) = \sum_{i=1}^n \log(f(y_i|\theta)) = \sum_i \ell(\theta|y_i)$
 - More desirable properties than $\mathcal{L}(\theta|y)$ (e.g., concave [remember Box-Cox], calculation

Maximum Likelihood Estimation (MLE)

- Find the values of the model parameters that maximize the likelihood function over the parameter space (i.e., the likelihood that the model produces the observed data)
- Log-likelihood is used in this process by solving its derivative with respect to θ set to 0
- Model with high or low log-likelihood?

Higher!

Akaike Information Criterion (AIC)

A model with p predictors

$$AIC = -2\ell(\hat{\theta}) + 2p$$

- For linear regression
 - $-2\ell(\hat{\theta}) = n \log \frac{RSS}{n} + \text{constant}$
 - Constant is the same for a given outcome variable and assumed error distribution (can be ignored when comparing models)
- Model fit = simplicity
- Better model has smaller or larger AIC?
- Bayesian information criterion (BIC)
 - $BIC = -2\ell(\hat{\theta}) + p \log(n)$

Akaike Information Criterion (AIC)

A model with p predictors

$$AIC = -2\ell(\hat{\theta}) + 2p$$

- For linear regression
 - $-2\ell(\hat{\theta}) = n \log \frac{RSS}{n} + \text{constant}$
 - Constant is the same for a given outcome variable and assumed error distribution (can be ignored when comparing models)
- Model fit = simplicity
- Better model has smaller or larger AIC?
- Bayesian information criterion (BIC)
 - $BIC = -2\ell(\hat{\theta}) + p \log(n)$

Smaller!

Adjusted R^2

- $R^2 = \frac{SS_{reg}}{SS_y} = 1 - \frac{RSS}{SS_Y}$
- $R_a^2 = 1 - \frac{RSS/(n-p)}{SS_Y/(n-1)} = 1 - \frac{\hat{\sigma}_{model}^2}{\hat{\sigma}_{null}^2} = 1 - \frac{n-1}{n-p} (1 - R^2)$
- Want larger or smaller?

Mallow's C_p

- $C_p = \frac{RSS_p}{\hat{\sigma}^2} + 2p - n$, where
 - RSS_p : RSS of the reduced model with p parameters
 - n : Sample size
 - $\hat{\sigma}^2$: MSE of the full model with k predictors $\hat{\sigma}^2 = RSS_k/(n - k)$
- $C_p = \frac{RSS_p - RSS_k}{\hat{\sigma}^2} + p - (k - p) = (k - p)(F_p - 1) + p$,

Where $F_p = \frac{(RSS_p - RSS_k)/((n - p) - (n - k))}{RSS_k/(n - k)}$

- C_p of full model = p
- C_p if p predictor model fits $\approx p$
- C_p around or less than p desirable

Mallow's C_p

- $C_p = \frac{RSS_p}{\hat{\sigma}^2} + 2p - n$, where
 - RSS_p : RSS of the reduced model with p parameters
 - n : Sample size
 - $\hat{\sigma}^2$: MSE of the full model with k predictors $\hat{\sigma}^2 = RSS_k / (n - k)$
- $C_p = \frac{RSS_p - RSS_k}{\hat{\sigma}^2} + p - (k - p) = (k - p)(F_p - 1) + p$

Want larger or smaller?

Where $F_p = \frac{(RSS_p - RSS_k) / ((n - p) - (n - k))}{RSS_k / (n - k)}$

- C_p of full model = p
- C_p if p predictor model fits $\approx p$
- C_p around or less than p desirable

Mallow's C_p

- $C_p = \frac{RSS_p}{\hat{\sigma}^2} + 2p - n$, where
 - RSS_p : RSS of the reduced model with p parameters
 - n : Sample size
 - $\hat{\sigma}^2$: MSE of the full model with k predictors $\hat{\sigma}^2 = RSS_k / (n - k)$
- $C_p = \frac{RSS_p - RSS_k}{\hat{\sigma}^2} + p - (k - p) = (k - p)(F_p - 1) + p$

Smaller!

Where $F_p = \frac{(RSS_p - RSS_k) / ((n - p) - (n - k))}{RSS_k / (n - k)}$

- C_p of full model = p
- C_p if p predictor model fits $\approx p$
- C_p around or less than p desirable



Forward, Backward, Stepwise Selection

Forward Selection

1. Start with a model only with an intercept as a running model
2. Choose the most significant predictor by running simple regressions and include it to the running model
3. Add one predictor among the remainder to the running model and choose the most significant set of two predictors
4. Continue until no additional predictors make a significant contribution to the model

Or replace predictor significance with AIC

Backward Selection

1. Start with the full model
2. Eliminate the least significant predictor and create a reduced model
3. Continue to eliminate the least significant predictor from reduced models
4. Continue until all variables in the model are significant

Or replace predictor significance

Stepwise Selection

Combination of Forward and Backward Selection

1. Start with an intercept only model
2. Choose the most significant predictor and add it to the model
3. Add each predictor among those remaining (one at a time) to the model and choose the most significant set of two predictors
4. Check if any variables become non-significant. If any are, then eliminate one at a time as in Backward selection
5. When no more variables can be eliminated, add variables to the model

Or replace predictor significance with AIC



Why might these methods be problematic?

Why might these methods be problematic?

- Summary of the problem: $E(\hat{\beta} | \hat{\beta} \text{ is significant}) \neq \beta$ ([Chatfield, 1995](#))
- Biased R^2 , $\hat{\beta}$, standard errors, and p-values
- F and χ^2 statistics do not have the claimed distribution² ([Grambsch & O'Brien, 1991](#))
- Will result in residual confounding if use variable selection to find confounders ([Greenland, 2000](#))
- Derksen & Keselman ([1992](#)) found that in stepwise analyses the final model represented noise 0.20-0.74 of time, final model usually contained $<1/2$ actual number of authentic predictors

Example: Issues with Technical Selection Methods

```
set.seed(13)
dat <- data.frame(matrix(data = rnorm(2100, 0,1),
                        nrow = 100, ncol = 21))
colnames(dat) <- c("x1", "x2", "x3", "x4", "x5",
                  "x6", "x7", "x8", "x9", "x10",
                  "x11", "x12", "x13", "x14", "x15",
                  "x16", "x17", "x18", "x19", "x20", "y")

dat.mod <- lm(y ~ ., dat)

summary(stepAIC(dat.mod, direction = "both"))
```

What do you expect to see in dat.mod?

Lab

What should we learn today?

- Variable transformation
 - How to find proper transformation for predictors
 - How to find proper transformation for the outcome
 - How to fit a polynomial regression
 - How do we interpret results when using variable transformation
 - How to fit a GAM
- Technical model selection
 - Criterion-based: AIC, Adjusted R^2 , BIC, Mallow's C_p